WEEK 4 · 23–26 NOVEMBER 2026

Implementation and testing

From a notebook that works today to a product someone can run next month

Week 4 · Session 2 · practical: 31_from_notebook_to_product.ipynb

LEARNING OBJECTIVES

What you will be able to do

- Move analysis code out of a notebook into a **module** the notebook, a CLI and an app all share
- Write **provenance and quality flags** next to every result, so it can be audited later
- Choose and document a decision **threshold** as a product decision, not a hidden constant
- Write **tests** that are worth having: unit tests, round-trips and product smoke tests
- Ship the result through a CLI, a small web app, or an API — and know which to pick
- **Profile** the pipeline and optimise the step that actually costs time

1. What makes it a product

The four things a notebook cannot do

THE DISTINCTION

A notebook proves the method can work.
A product is what someone else can run, trust and act on.

Notebooks cannot be imported, scheduled, tested or reviewed line by line — and the execution order is invisible in the file.

The shape of the example product

- `app/mangrove_product.py` — `compute_features()`, `run_pipeline()`, `main()` — **the product**
- `app/streamlit_app.py` — a viewer for what the pipeline wrote — **the delivery**
- `tests/test_hsi_utils.py` — 23 automated checks — **the safety net**
- `hsi_utils.py` — shared I/O and spectral helpers — **the library**
- The notebook imports the module, exactly as the CLI and the app do
  - Rule of thumb: **the notebook tells the story, the module does the work**

The example: mangrove extent from Tanager

- Six named features, not 352 bands: NDVI, NDWI, NDMI, NDRE, brightness, SWIR ratio
- Small random forest, trained on WorldCover pure pixels with a **spatially blocked** split
- Test accuracy **0.984**; mangrove class F1 **0.975** (precision 0.983, recall 0.968)
- Few, named features keep the model fast, small and **explainable to a user**
- Post-processing: probability threshold, then a minimum mapping unit of 5 pixels (~0.45 ha)

2. Provenance and decisions

Making a result auditable

EVERY RUN, EVERY TIME

What run_pipeline writes next to the data

```
outputs/31/<run_id>/
    mangrove.tif        # 2-band COG: mask + probability, georeferenced
    quicklook.png       # what a human looks at first
    metadata.json       # parameters, model, versions, bbox, attribution
    report.html         # one page a non-specialist can read
```

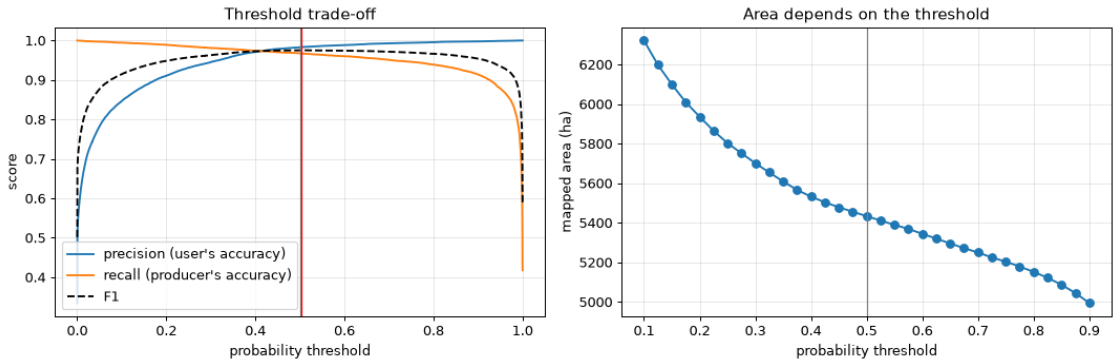
A number without its parameters and date cannot be audited

Quality flags: let the run tell you not to trust it

- `LOW_COVERAGE` — more than 40 % of the window is cloud or no-data
- `NO_DETECTION` — nothing found; the user must know the difference between zero and failure
- `CLOUD_MASK_OFF` — run made without the cloud mask (remember the sabkha false positives)
- `OK` — all checks passed
- Flags are **explicit rules in code**, not a paragraph in a report nobody reads

The threshold is a product decision

- Precision and recall trade against each other as the threshold moves
- Best F1 was **0.977 at 0.52** — but the product ships **0.50**
- Why: a round number is easier to explain, document and reuse
- Mapped area changes steeply with the threshold — show the curve, not one number
- Write the choice and the reason into the metadata



Threshold trade-off and the resulting mapped area

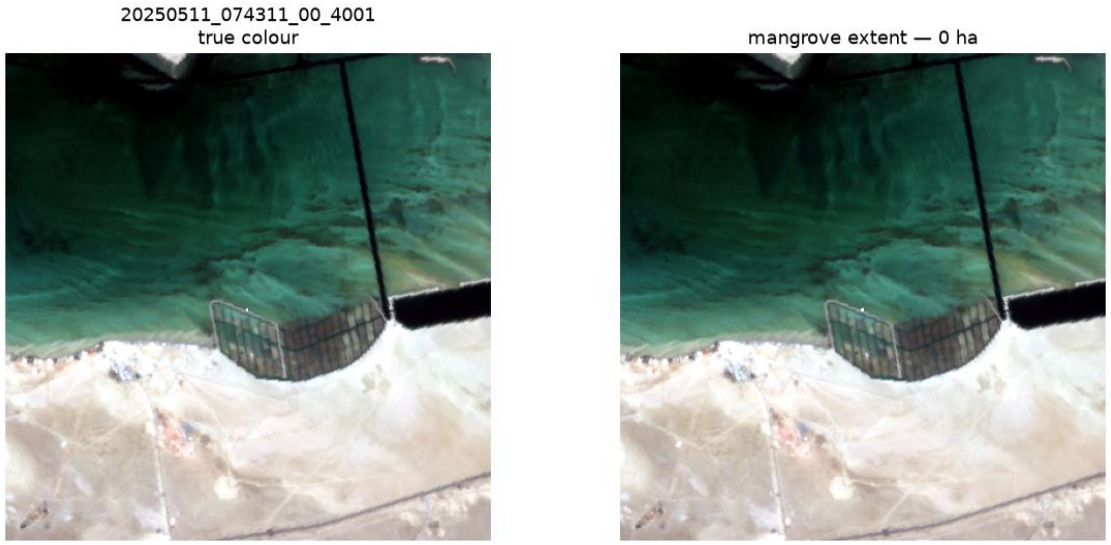
One call, one auditable result





5 386 ha of mangrove mapped in about one second, with flags OK

The same product somewhere else



Domain-shift run over the UAE coast

- Run the El Salvador model on the Al Dhafra coast, UAE
- Result: **0 ha**, flag `NO_DETECTION`, 60.6 % usable pixels
- It did **not** hallucinate mangroves on bright sabkha — the failure mode we feared
- WorldCover also maps no mangrove in this window, so 'nothing here' is defensible
- But absence of detection is **not** evidence of absence — say where you looked

3. Testing

How you know you did not break it

Two kinds of test, both cheap

Kind	Question it answers	Example
Unit test	Does this function still do what it promises?	nd(3, 1) == 0.5; good_band_mask drops 1400 nm
Round-trip	Does writing then reading change the data?	A subset returns the same values and grid
Smoke test	Does a whole run still produce something sane?	Outputs exist, CRS set, area within bounds
Regression	Did today's change move the answer?	Area within ±5 % of a stored value

What to test, and what not to

- ✓ Pure functions with known answers: indices, band picking, unit conversions, geometry
- ✓ Round-trips: write then read a GeoTIFF or subset and compare
- ✓ Invariants: shapes, ranges, CRS present, no silent all-NaN output
- ✓ A tiny regression fixture: one small window, a stored expected area with a tolerance
- ✗ Exact floating-point equality of model accuracies — they move with library versions
- ✗ Anything needing a 1 GB download in CI — mark it slow or skip without local data

A product smoke test in six assertions

```
def smoke_test(meta):
    run = hu.OUTPUT_DIR / "31" / meta["run_id"]
    for name in ("mangrove.tif", "metadata.json", "quicklook.png", "report.html"):
        assert (run / name).exists(), f"missing output: {name}"
    with rasterio.open(run / "mangrove.tif") as src:
        assert src.crs is not None and src.transform.a > 0   # georeferenced
        mask, prob = src.read(1), src.read(2)
    assert set(np.unique(mask)) <= {0, 1}                     # binary mask
    assert 0 <= prob.max() <= 100                             # probability in per cent
    assert 0 <= meta["results"]["mangrove_area_ha"] <= scene_area
    assert meta["attribution"].startswith("Tanager")          # credit present
```

If a future change breaks one of these, you find out immediately

4. Delivery and performance

Getting it to the user, fast enough

The command line makes it schedulable

- `python app/mangrove_product.py --scene <id> --window 100 500 100 500`
- No manual steps, no interactive input, a proper exit code and log output
- A monthly cron entry is then a one-liner
- For the PoC you do not need the cron job — you need to show it **could** be scheduled
  - The notebook runs the CLI with subprocess so you can see it work

Choosing a delivery route

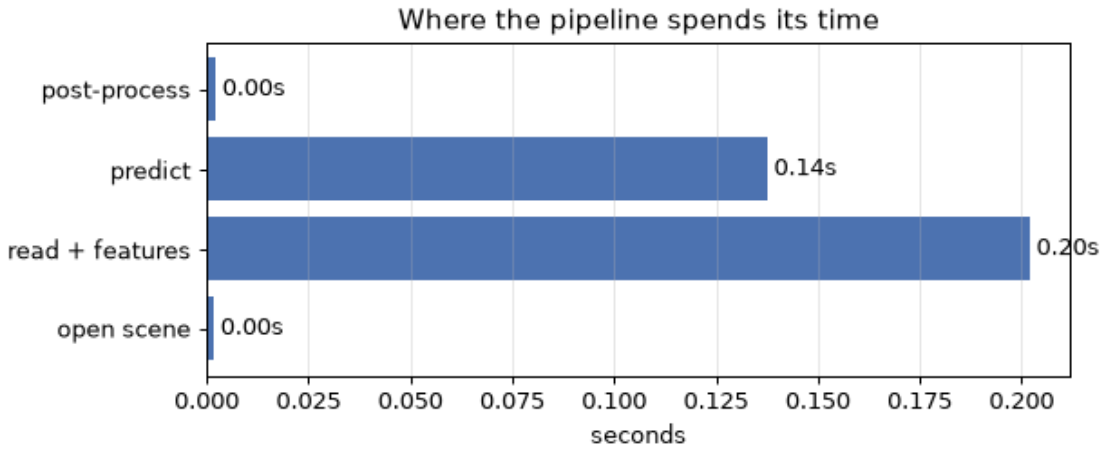
Route	When it fits	Effort
Static COG + report on a shared drive	The user already has GIS software	Lowest
Streamlit or Dash app	Non-technical users need to look and filter	Low
Web map (COG + tiles)	Results must sit on other layers	Medium
REST API	Another system consumes the result	Medium
Alerting (email/SMS on a threshold)	The user only cares about change	Medium

The Streamlit viewer, in one command

- `streamlit run app/streamlit_app.py` — reads whatever runs exist in `outputs/31/`
- Shows the quicklook, the mask, the probability layer and the metadata
- An **interactive threshold slider** so the user sees the sensitivity themselves
- Pixel inspector with the full Tanager spectrum, if the scene HDF5 is available
- A download button for the GeoTIFF — the user leaves with the data, not a screenshot

Profiling: optimise the step that actually costs

- Almost all the time is **I/O** — decompressing HDF5 chunks — not maths
- **Read once, reuse**: one pass for all bands beats repeated `read_band` calls
- **Read less**: a window or stride beats the full scene
- **Tile** full scenes so memory stays bounded (notebook 30)
- Only then tune the model: the forest is milliseconds per 10^5 pixels



Time per pipeline stage, and the cProfile top entries

Handover checklist

- `pip install -r requirements.txt` then **one documented command** reproduces the outputs
- No absolute paths, no manual steps, no cells that must run out of order
- Every output carries metadata: parameters, versions, scene IDs, date, attribution
- Quality flags tell the user when a result is not trustworthy
- Tests pass, and it is obvious how to run them (`pytest tests/ -q`)
- A user-facing view exists and **someone outside the team has tried it**
- `templates/submission_checklist.md` is fully ticked

PRACTICAL

Over to you

▶ `31_from_notebook_to_product.ipynb` · `120 min`

- Sections 1–2: read the module, train the model, choose and justify your threshold
- Section 3: run the pipeline twice — your area, and somewhere it should fail
- Section 4: run `pytest tests/ -q`, then write a smoke test for your own product
- Sections 5–6: run the CLI, then start the Streamlit viewer
- Section 7: profile your own pipeline and fix the slowest step

KEY TAKEAWAYS

Recap

- The notebook tells the story; an importable **module** does the work
- Ship **provenance and quality flags** with every result
- The **threshold** is a documented product decision, not a hidden constant
- **Tests** are cheap for pure functions and round-trips — and they let you change code confidently
- Pick the **simplest delivery** that reaches the user; a reliable COG beats a broken dashboard
- Profile first: this pipeline is **I/O-bound**, so read once, read less, tile

Next: Session 3 — Presentation preparation and project rehearsal

References and further reading

Wilson, G. et al. (2017). Good enough practices in scientific computing. PLoS Computational Biology 13(6), e1005510.

Rule, A. et al. (2019). Ten simple rules for writing and sharing computational analyses in Jupyter notebooks. PLoS Comp. Biol . 15(7), e1007007.

Cloud Optimised GeoTIFF specification — cogeo.org

Course materials: `templates/submission_checklist.md`, `templates/evaluation_rubric.md`, `tests/test_hsi_utils.py`

Tanager open data CC-BY-4.0 © Planet Labs PBC · ESA WorldCover 2021 v200 (CC-BY 4.0)